

COMPLETE PROJECT HANDBOOK

IntelliQR

Architecture, feature implementation, file structure, security, storage, deployment, commands, testing, and operational guide.

Current implementation reference · August 2026

Frontend: React + TypeScript + Vite

Backend: FastAPI + MongoDB

Production: Vercel + Render + Atlas + Vercel Blob + Brevo

Contents

1. Project purpose and scope
2. System architecture and request flow
3. Technology stack and design choices
4. Frontend structure and routing
5. Backend structure and layered design
6. Database collections and data ownership
7. Authentication and session restoration
8. QR Generator and BrandCraft
9. Dynamic QR tracking and analytics
10. SafeScan QR scanner
11. SecureVault protected destinations
12. ShareVault encrypted file sharing
13. BulkForge batch generation
14. Dashboard, profile, admin and notifications
15. Internationalization, theme and responsive UI
16. Security and privacy model
17. Storage strategy and deployment
18. Environment variables and start commands
19. Testing and troubleshooting
20. Limitations and future improvements

Document scope. This handbook describes the current repository and deployed design. Secrets and actual credential values are intentionally excluded.

1. Project purpose and scope

IntelliQR is a full-stack QR management platform. It does more than draw a QR image: it creates multiple payload types, persists user-owned records, supports editable tracked links, analyzes scans, protects destinations and files, generates batches, records analytics, sends notifications, and provides administrative controls.

Main user capabilities

- Register, sign in, restore a session and manage a profile.
- Create URL, text, email, phone, Wi-Fi, contact and location QR codes.
- Customize colors, gradients, modules, frames, correction level, size, margin and logo.
- Download PNG, SVG or PDF and manage saved QR records.
- Decode uploaded/camera QR images and assess suspicious URLs.
- View real dynamic-scan analytics.

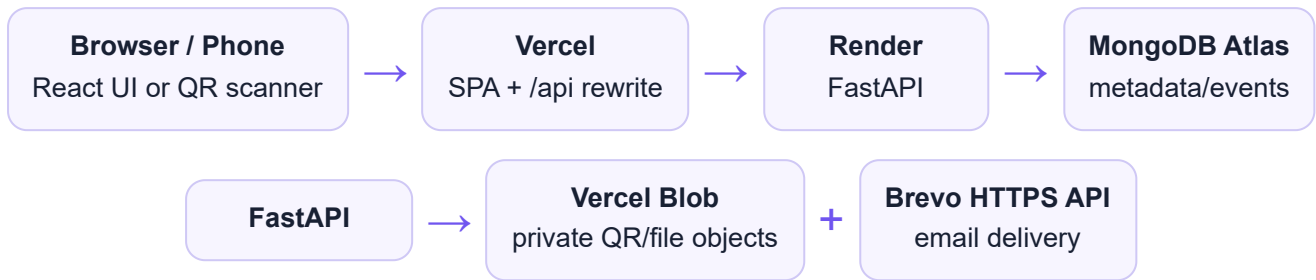
Advanced capabilities

- SecureVault policies for protected dynamic destinations.
- ShareVault AES-GCM encrypted file distribution.
- BulkForge CSV/XLSX imports and ZIP exports.
- In-app and Brevo email notifications.
- Role-based admin dashboard and audit history.
- English, Hindi and Gujarati interface support.

Quality goals

The implementation is owner-isolated, typed end-to-end, privacy-aware, responsive, testable, deployable on free-tier services, and designed to keep secrets out of the browser and repository.

2. System architecture



Why the frontend uses a Vercel rewrite

The browser calls relative paths under `/api/v1`. Vercel rewrites `/api/*` to Render. This produces same-origin browser requests, simplifies CORS, and allows the HttpOnly refresh cookie to work reliably. A second rewrite sends arbitrary application paths to `index.html`, allowing refreshes on `/dashboard`, `/share/:slug`, and other React Router routes.

Typical authenticated request

1. A page calls a feature API module.
2. The shared API client attaches the in-memory Bearer access token.
3. FastAPI validates the token and resolves the current user.
4. The endpoint calls a service; the service applies business rules.
5. A repository reads or writes MongoDB, and infrastructure adapters handle files, QR rendering or email.
6. The response schema is returned to the typed React hook and UI.

Public QR request

A dynamic QR contains `https://intelijqr-backend.onrender.com/r/{slug}`. Render atomically records the scan and redirects to the current destination. A ShareVault QR instead opens `https://intelij-qr.vercel.app/share/{slug}` directly; its download event is recorded when authorization succeeds.

3. Technology stack

Layer	Technology	Purpose and reason
UI	React 19, TypeScript	Component-based interface with compile-time contracts.
Build	Vite 7	Fast development server and optimized production bundles.
Styling	Tailwind CSS 4, custom CSS	Responsive layout, light/dark themes and reusable visual tokens.
Routing	React Router 7	Public, protected, role-restricted and parameterized pages.
Icons/UI	Lucide, Radix, reusable components	Accessible primitives and consistent cards, buttons, inputs, badges.
API	FastAPI, Uvicorn	Async-friendly typed REST endpoints and automatic OpenAPI docs.
Validation	Pydantic v2	Request, response and environment validation.
Database	MongoDB, PyMongo Async	Flexible documents, indexes and aggregation pipelines.
QR	qrcode, Pillow	Matrix generation and PNG/SVG/PDF rendering.
Decode	OpenCV, pyzbar	Image decoding and multi-code support.
Security	PyJWT, bcrypt/pwdlib, cryptography	JWT sessions, password hashing and AES-256-GCM encryption.
Files	Vercel Blob	Private durable object storage outside Render's temporary disk.
Email	Brevo REST API through httpx	HTTPS transactional delivery because free Render blocks SMTP ports.
Hosting	Vercel, Render, Atlas	Separate frontend/API/data deployment with free-tier support.
Tests	pytest, Vitest	Backend behavior, security, storage and frontend API contracts.

Virtual environment. `backend/.venv` isolates Python packages from system Python, keeps versions reproducible, and prevents dependency conflicts. It is local development tooling and is not committed or copied to Render.

4. Repository and file structure

IntelijQR/	
├─ command.txt	Windows start/stop reference
├─ docs/	generated handbook and viva PDFs
├─ frontend/	
│ └─ src/App.tsx	route table and lazy page loading
│ └─ src/pages/	screen-level components
│ └─ src/features/	feature API, hooks, types, components
│ └─ src/components/	layouts, protected routes, shared UI
│ └─ src/lib/api/client.ts	common fetch/token/refresh behavior
│ └─ src/hooks/	theme and backend status
│ └─ src/index.css	global themes and responsive styling
│ └─ tests/	Vitest API-contract tests
│ └─ vite.config.ts	Vite/Tailwind/dev proxy
│ └─ vercel.json	production API and SPA rewrites
└─ backend/	
│ └─ app/main.py	FastAPI app, middleware, errors, lifespan
│ └─ app/api/	dependency injection and HTTP endpoints
│ └─ app/core/	config, logging, exceptions, crypto
│ └─ app/db/mongodb.py	connection and index creation
│ └─ app/models/	domain dataclasses
│ └─ app/schemas/	Pydantic request/response contracts
│ └─ app/repositories/	MongoDB persistence and aggregation
│ └─ app/services/	business logic and workflows
│ └─ app/infrastructure/	QR, storage, bulk and email adapters
│ └─ tests/	pytest suite
│ └─ scripts/	integration smoke tests
│ └─ requirements.txt	Python dependencies
│ └─ .env.example	safe configuration template

Frontend organization

Each feature folder normally contains `api/` for HTTP calls, `hooks/` for stateful orchestration, `types/` for TypeScript contracts, and optional feature components. Pages compose these pieces; shared visual components do not know business rules.

Backend organization

Endpoints translate HTTP to application calls. Services enforce rules. Repositories own database queries. Infrastructure adapters talk to external systems. Schemas validate boundary data, while domain models express stored application state. `dependencies.py` wires these layers together.

5. Frontend routes and behavior

Route	Access	Responsibility
/	Public	Landing page and product introduction.
/login , /register	Public	Authentication forms and errors.
/access/:slug	Public entry	SecureVault password/member authorization.
/share/:slug	Public entry	ShareVault policy display, grant and download.
/dashboard	Signed in	Summary, history, QR management and edit actions.
/generator	Signed in	Payload forms, BrandCraft preview and exports.
/scanner	Signed in	Image/camera decode, classification and SafeScan.
/analytics	Signed in	Period filters, scan trends and breakdowns.
/bulk	Signed in	CSV/XLSX import, job polling and ZIP download.
/share-vault	Signed in	Encrypted upload, policies, QR and history.
/profile	Signed in	Name, password, locale and settings.
/notifications	Signed in	Inbox and channel/category preferences.
/admin	Admin	Platform metrics, users and audit controls.

Session restoration on refresh

The access token stays only in JavaScript memory. On a full refresh it disappears, so `AuthContext` calls the refresh endpoint. The browser automatically supplies the `HttpOnly` refresh cookie; the backend rotates it and returns a new access token and user. `ProtectedRoute` waits for initialization before deciding whether to render or redirect, preventing the previous “refresh returns to registration/login” bug.

Backend status indicator

The shared top bar uses `useBackendStatus` to call the health endpoint. Its attractive Online/Offline state appears across authenticated pages. This indicates API reachability, not the health of every third-party dependency.

6. Backend API and request pipeline

Application startup

1. `uvicorn app.main:app` imports the FastAPI application.
2. Pydantic Settings loads `backend/.env` locally or service environment variables in production.
3. The lifespan handler connects to MongoDB and creates indexes.
4. CORS, request-ID and security headers are applied.
5. Versioned routers are mounted at `/api/v1` ; dynamic redirects are mounted at `/r` .

Endpoint groups

Prefix	Functions
<code>/api/v1/auth</code>	Register, login, refresh and logout.
<code>/api/v1/users</code>	Profile, password and locale.
<code>/api/v1/qr/generations</code>	Generate, list, edit, delete, favorite and download.
<code>/api/v1/qr/scans</code>	Decode, analyze, list and delete scan history.
<code>/api/v1/analytics</code>	Owner-scoped trends and breakdowns.
<code>/api/v1/qr/access</code>	SecureVault policy and authorization grants.
<code>/api/v1/shares</code>	ShareVault CRUD, public policy, grants, download and history.
<code>/api/v1/bulk</code>	Batch jobs, progress, ZIP download and deletion.
<code>/api/v1/notifications</code>	Inbox, unread state and preferences.
<code>/api/v1/admin</code>	Admin statistics, user controls and audits.
<code>/api/v1/health</code>	Liveness/readiness checks.

Error contract

Application exceptions return a stable object containing `code` , user-safe `message` and `request_id` . Validation errors return HTTP 422 with details. Unexpected failures are logged server-side and return a generic 500 response without leaking internals.

7. Database design

Collection	Stored information
users	Email, display name, password hash, role, status, locale, token version and timestamps.
sessions	Hashed/identified refresh sessions, expiry and revocation state.
qr_generations	Owner, payload summary, encrypted destination where needed, design, slug, file paths, limits and scan count.
qr_scan_events	Dynamic redirect analytics with privacy-safe visitor and device context.
qr_scans	Decoded scanner content, type, risk analysis and source; not the uploaded image.
share_files	Encrypted object path, content hash, policy, slug, counts and lifecycle.
share_download_events	Download time and privacy-safe client context.
bulk_jobs	Input/job state, formats, progress, bounded errors and ZIP reference.
notifications	User event inbox and read state.
notification_preferences	In-app/email and category choices.
download_events	Lightweight export history used on Dashboard.
admin_audit_events	Who changed which account and when.

Indexes and ownership

Startup creates unique email and slug constraints plus owner/time, generation/time, visitor/time and unread indexes. Protected queries combine resource ID with the authenticated `user_id`, preventing one user from reading or changing another user's resources.

Why MongoDB? Feature records contain different nested structures—QR payload details, design dictionaries, analytics dimensions and policy fields. MongoDB provides flexible documents while still supporting validation in Pydantic, unique indexes and aggregation pipelines.

8. Authentication and security sessions

Registration and login

Email is normalized, duplicates are rejected, and passwords are hashed with bcrypt through `pwdlib`. Login verifies the hash and account status. Admin role is assigned only when the normalized email matches the configured allowlist.

Two-token model

- **Access token:** short-lived JWT returned in JSON and held only in memory. It authorizes API calls.
- **Refresh token:** longer-lived token stored in a Secure, HttpOnly cookie in production. JavaScript cannot read it.

Refresh rotation limits replay: a refresh request replaces the old session/token. Logout revokes the session and clears the cookie. Password changes increment `token_version` and revoke all sessions, forcing every device to authenticate again.

Authorization

`get_current_user` resolves Bearer credentials. Role-aware protected routes and backend admin dependencies enforce admin access on both UI and API layers. Hiding an admin link alone would not be security; the backend check is authoritative.

9. QR Generator and BrandCraft

Supported payloads

Type	Encoded standard
URL	HTTP/HTTPS destination, direct or dynamic.
Text	UTF-8 plain text.
Email	<code>mailto:</code> with optional subject/body query.
Phone	<code>tel:</code> URI.
Wi-Fi	Wi-Fi configuration syntax with escaped fields.
Contact	vCard fields.
Location	<code>geo:</code> coordinates and label.

Generation workflow

1. The React form builds a typed request and live preview.
2. Pydantic validates content, policy and BrandCraft design.
3. `QrPayloadBuilder` creates a standards-compatible payload.
4. For dynamic URLs, the renderer receives the stable Render `/r/{slug}` link. Direct URLs and non-URL types encode their payload directly.
5. `QrRenderer` creates one matrix, then PNG, SVG and PDF outputs.
6. Private Vercel Blob stores artifacts at `{storageKey}/qr.png|svg|pdf` ; MongoDB stores their paths and metadata.
7. The API returns download routes; authenticated downloads stream bytes with a safe filename.

BrandCraft

The design contract stores foreground/background colors, optional gradient and direction, square/rounded/dot modules, frame shape/text, correction level L/M/Q/H, size, quiet-zone margin and validated PNG/JPEG logo. Contrast validation protects scan reliability. The logo is normalized, size-limited and placed over a background plate.

Operational detail. QR records created before durable Blob storage may reference Render files already erased by a redeploy. Those artifacts cannot be recovered and must be regenerated.

10. Dynamic tracking and Analytics

Direct versus dynamic

Direct/static	Dynamic/tracked
QR contains the final destination.	QR contains the backend <code>/r/{slug}</code> .
No server is required after creation.	Backend must remain reachable.
Destination cannot be changed.	Destination can change without reprinting.
No redirect analytics.	Scans are recorded before redirect.

Tracking sequence



The analytics service stores time, generation ID, device category, browser, operating system, approximate locality and a keyed one-way visitor hash. Raw IP addresses and complete user-agent strings are not retained. Public country/city remains Unknown unless a GeoIP database is configured.

Analytics output

Period filters provide total scans, unique visitors, comparison percentage, time-series points, device/browser/OS/country/city breakdowns, top QR codes and recent events. All queries are restricted to the authenticated owner.

11. SafeScan scanner

SafeScan accepts PNG, JPEG and WebP images up to 10 MB from upload or a captured webcam frame. OpenCV detects and decodes one or multiple QR symbols. The scanner does not store the uploaded image.

Processing pipeline

1. Validate file type and byte limit.
2. Decode symbols with the image decoder.
3. Classify decoded content as website, email, contact, phone, Wi-Fi, payment or text.
4. For URLs, run transparent local heuristics such as suspicious scheme/host patterns.
5. Persist only decoded content and analysis metadata in owner history.
6. If a result is risky and the user opted in, create an in-app/email security alert.

SafeScan is an aid, not antivirus. Heuristics can detect warning signs but cannot guarantee that a URL is safe. The product clearly presents this limitation.

12. SecureVault

SecureVault protects the destination behind a dynamic URL QR. It is distinct from ShareVault: SecureVault controls a URL redirect; ShareVault distributes an uploaded file.

Policy	Access rule
Public	Redirect immediately.
Password	Visitor enters a shared password of at least eight characters.
Members	Visitor must sign in.
Private	Signed-in email must appear in the allowlist.

Protection flow

Protected destinations are encrypted with AES-256-GCM and a unique nonce. Passwords are bcrypt hashes, never reversible encryption. Scanning opens `/access/:slug`. After policy approval, the backend issues a short-lived purpose-bound JWT grant tied to the slug and user/visitor. The redirect validates the grant and uses no-store/no-referrer headers before forwarding.

13. ShareVault

What it does

ShareVault uploads approved documents, images, videos and text files, encrypts the bytes, stores them privately, creates a public access link and QR, applies access rules, and records successful downloads.

Upload workflow

1. Validate size (default 25 MB), extension and basic file signature.
2. Create a random slug and storage key.
3. Encrypt bytes locally in the backend using AES-GCM.
4. Upload encrypted bytes to private Vercel Blob as `sharevault/{key}.vault`.
5. Create a direct QR to `https://intellij-qr.vercel.app/share/{slug}`.
6. Persist metadata, policy and object path in MongoDB.

Download workflow

1. The phone opens the Vercel access page from the QR.
2. The page loads the public policy; authentication/password UI appears when required.
3. The grant endpoint verifies the policy and returns a short-lived grant.
4. The download endpoint revalidates the grant, status, account and atomic download limit.
5. The encrypted object is fetched and decrypted in memory.
6. The file response is sent with attachment, no-store and nosniff headers.
7. A privacy-safe download event and optional email notification are created.

Pausing, expiry or reaching the maximum download count makes the share unavailable. Deleting a share removes the Blob, download events and paired QR record.

14. BulkForge

BulkForge imports CSV or XLSX (up to 5 MB and normally 250 rows), validates each row independently, reuses the same QR generator, and produces a ZIP containing selected PNG/SVG/PDF outputs.

Why reuse the generator service?

It prevents two definitions of a valid QR. Individual and batch jobs share payload validation, rendering, storage and persistence. Invalid rows are isolated so one bad row does not necessarily fail the complete job.

Job states

queued processing completed partial failed

The frontend polls persisted job status. The backend bounds reported validation errors, packages in-memory/cloud-fetched QR bytes into a safe ZIP, and sends an in-app/email result notification. The ZIP itself currently uses configured bulk storage; on ephemeral Render storage, long-term ZIP retention remains a future improvement.

15. Dashboard, profile, admin and notifications

Dashboard

Repository aggregation combines QR counts/favorites, dynamic scans, scanner history, shared files/downloads, BulkForge jobs and exports. QR management supports search, filters, favorites, pause/activate, editing, downloading and deletion.

Profile

Users edit display name and locale. Password rotation verifies the old password, hashes the new one, revokes sessions and forces a clean re-login. Theme choice is a local browser preference.

Administration

Configured administrators see aggregate statistics, searchable/paginated accounts, role/status controls and audit history. Server-side rules prevent self-lockout and removal of the final active administrator. Account access changes invalidate tokens and sessions.

Notifications and email

Events include risky scans, QR limit reached, ShareVault downloads and BulkForge completion. Users choose in-app/email channels and category preferences. Brevo delivery uses the HTTPS endpoint `/v3/smtp/email`, not SMTP, because free Render blocks outbound ports 25/465/587. The API key remains only in Render. Emails contain branded responsive HTML plus a plain-text fallback and escaped user-controlled content.

16. Internationalization, theme and responsive UI

`LocaleContext` supplies English, Hindi and Gujarati dictionaries through `useLocale`. The selected locale is saved to the account and survives session restoration. QR content remains Unicode/UTF-8 and has render/decode coverage for multiple scripts.

`useTheme` controls light/dark mode, while CSS selectors under `html[data-theme]` provide theme-specific surfaces, borders, text and scrollbars. Shared layouts make the sidebar, top bar and content areas consistent. Generator columns use responsive constraints; the preview participates in scrolling and avoids a zoom-specific fixed layout.

Accessibility considerations include labeled controls, semantic buttons, visible focus states, error roles, meaningful status text, color contrast validation for QR output, and responsive tables/scroll regions.

17. Security and privacy model

Risk	Control
Password disclosure	Bcrypt hashes; secrets excluded from responses and Git.
Token theft	Short access lifetime, HttpOnly refresh cookie, rotation and revocation.
Cross-user access	Owner ID included in protected repository queries.
File disclosure	AES-GCM before private Blob upload; short-lived grants.
Destination disclosure	AES-GCM for protected URLs.
Path traversal	Random keys, safe names and resolved-path checks.
Malicious upload	Extension, size and signature checks; no antivirus claim.
HTML injection in email	All notification title/body content is escaped.
Information leakage	Generic 500s; request IDs; no raw IP retention.
CORS/cookie misuse	Explicit production origin, Secure cookie and same-origin rewrite.
QR unreadability	Contrast validation, margin and correction-level controls.

Secret rotation. MongoDB credentials, JWT secret, Vault key, Blob token and Brevo key must never appear in source control, screenshots or documentation. If exposed, revoke/rotate them immediately.

18. Storage and production deployment

Data	Production location	Persistence
Accounts, QR metadata, events, policies	MongoDB Atlas	Durable database
ShareVault encrypted files	Private Vercel Blob	Durable object
QR PNG/SVG/PDF and logo	Private Vercel Blob	Durable object
Bulk ZIP archive	Configured bulk filesystem	Ephemeral on free Render
Frontend assets	Vercel CDN	Built from Git
Secrets	Render/Vercel environment settings	Platform-managed

Production URLs

- Frontend: `https://intelij-qr.vercel.app`
- Backend: `https://intelijqr-backend.onrender.com`
- Dynamic base: exactly `https://intelijqr-backend.onrender.com` (spelling matters).

Deployment sequence

1. Push `main` to GitHub.
2. Render builds backend requirements and runs Uvicorn with `--host 0.0.0.0 --port $PORT`.
3. Vercel builds the `frontend` Vite project and applies `vercel.json`.
4. Render receives production environment variables and connects to Atlas, Blob and Brevo.
5. Run health, login, refresh, QR, tracking, ShareVault and email checks.

Free Render cold start. An inactive free service can sleep; the first request may take about 50 seconds. The frontend may briefly show Offline until the health call completes.

19. Environment variables

Variable	Production purpose
INTELLIQR_APP_ENV	production .
INTELLIQR_MONGODB_URI	Atlas connection secret.
INTELLIQR_MONGODB_DATABASE	Database name.
INTELLIQR_MONGODB_REQUIRED_ON_STARTUP	Fail startup when DB is unavailable.
INTELLIQR_JWT_SECRET	At least 32 random characters.
INTELLIQR_VAULT_ENCRYPTION_KEY	Stable AES key; changing it breaks old encrypted content.
INTELLIQR_COOKIE_SECURE	true under HTTPS.
INTELLIQR_CORS_ORIGINS	["https://intelij-qr.vercel.app"] .
INTELLIQR_FRONTEND_BASE_URL	https://intelij-qr.vercel.app .
INTELLIQR_REDIRECT_BASE_URL	https://intelijqr-backend.onrender.com .
INTELLIQR_BLOB_READ_WRITE_TOKEN	Private Vercel Blob token.
INTELLIQR_BREVO_API_KEY	Brevo HTTPS API secret.
INTELLIQR_NOTIFICATION_FROM_EMAIL	Verified Brevo sender.
INTELLIQR_NOTIFICATION_FROM_NAME	Displayed sender name.
INTELLIQR_ADMIN_EMAILS	JSON allowlist for administrator role.

20. Start, build and test commands

First-time backend setup

```
cd C:\codes\react\IntelijQR\backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env # then configure local values
```

Start backend

```
cd C:\codes\react\IntelijQR\backend
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Python entry module: `backend/app/main.py` . Import string: `app.main:app` . Swagger:
`http://127.0.0.1:8000/docs` .

Start frontend

```
cd C:\codes\react\IntelijQR\frontend
npm install
npm run dev -- --host 0.0.0.0
```

Frontend: `http://127.0.0.1:5173` . For same-Wi-Fi phone testing use the computer's LAN IP in both frontend and backend URLs/configuration.

Validation

```
# Backend
cd backend
.\.venv\Scripts\python.exe -m pytest

# Frontend
cd frontend
npm run typecheck
npm test
npm run build
```

Stop

Press `Ctrl+C` in each server terminal. If port 8000 is already occupied, identify the owning PID with `Get-NetTCPConnection -LocalPort 8000` and stop only that intended process.

21. Testing strategy

The backend suite currently covers authentication/security, schemas, QR payloads/rendering/storage, scanning, analytics, SecureVault, ShareVault, BulkForge, notifications/Brevo, dashboard/profile, admin and REST contracts. Storage adapters use fake clients for deterministic encrypted/durable round trips. The frontend tests verify feature API contracts, while TypeScript and the Vite production build catch interface and bundling errors.

Recommended production smoke test

1. Open health endpoint and wait through any cold start.
2. Register/login; refresh a protected page and confirm session restoration.
3. Create a new dynamic URL QR and decode it to verify the exact Render hostname.
4. Scan on another device; confirm redirect and analytics increment.
5. Download PNG/SVG/PDF; confirm new objects appear in Vercel Blob.
6. Create public/password/private ShareVault examples; scan, authorize and download.
7. Confirm download history, in-app alert and branded Brevo email.
8. Run a small BulkForge CSV with one valid and one invalid row.
9. Verify unauthorized users cannot access owner/admin resources.

22. Troubleshooting guide

Symptom	Likely cause and fix
Port 8000 already in use	Another backend is running. Stop that PID or use one server only.
Frontend says Offline	Render cold start, wrong API rewrite, or backend unavailable. Check health and logs.
Refresh returns to login	Refresh cookie/route configuration. Check Secure cookie, Vercel rewrite and session restore.
Phone QR shows Render Not Found	Decoded QR has wrong backend hostname. Correct <code>INTELLIQR_REDIRECT_BASE_URL</code> , redeploy and create a new QR.
Old QR file download is 404	Artifact was on old Render disk. Regenerate; new artifacts use Blob.
ShareVault Blob list empty	Wrong environment key. It must be <code>INTELLIQR_BLOB_READ_WRITE_TOKEN</code> .
Share requires login	It was created as Members/Private. Use Public or supply an allowed signed-in account.
No tracking analytics	QR is direct/static or contains an old/wrong redirect URL. Create dynamic mode and scan the newly generated image.
Email toggle disabled	Brevo key/from email missing or sender not verified.
SMTP fails on Render free	Outbound SMTP ports are blocked. Use the implemented Brevo HTTPS API.
MongoDB validation/start failure	Missing/malformed URI, database or required secrets.

23. Limitations and future improvements

- Free Render cold starts delay first requests.
- Bulk ZIP files still need durable object storage for long-term retention.
- GeoIP is intentionally not cloud-resolved; public location remains Unknown without a local database.
- SafeScan is heuristic and not a malware sandbox.
- Gmail as a Brevo sender lacks custom-domain DKIM/DMARC; production deliverability should use an owned domain.
- Old QR artifacts lost from Render cannot be reconstructed unless regenerated from stored metadata/design.
- Object lifecycle cleanup/reconciliation could detect database/blob drift.
- A job queue would improve large BulkForge workloads beyond background tasks.
- Rate limiting, CAPTCHA, MFA, audit export and automated end-to-end browser tests would strengthen production readiness.

Conclusion

IntelliQR demonstrates a complete full-stack system: typed UI, structured API, persistent data, secure sessions, QR standards, dynamic tracking, privacy-aware analytics, encryption, object storage, batch jobs, notifications, deployment and automated tests. Its strongest architectural decision is separation of concerns: React handles interaction, FastAPI enforces rules, MongoDB holds state, Vercel Blob holds durable bytes, and external secrets remain server-side.

End of IntelliQR Complete Project Handbook · Generated from the current repository.